

OpenClaw Camera Application Tutorial

Before starting, please complete [01-Peripheral Hardware Configuration], [02-OpenClaw API-KEY Configuration], [03-OpenClaw Model Switching], [04-OpenClaw Prompts], [05-AI Voice Interaction Configuration], and [06-Three Conversation Scheme Configuration and Testing] in the `Preparation for Use` directory. This document assumes the basic environment is already prepared and only expands on camera-related content.

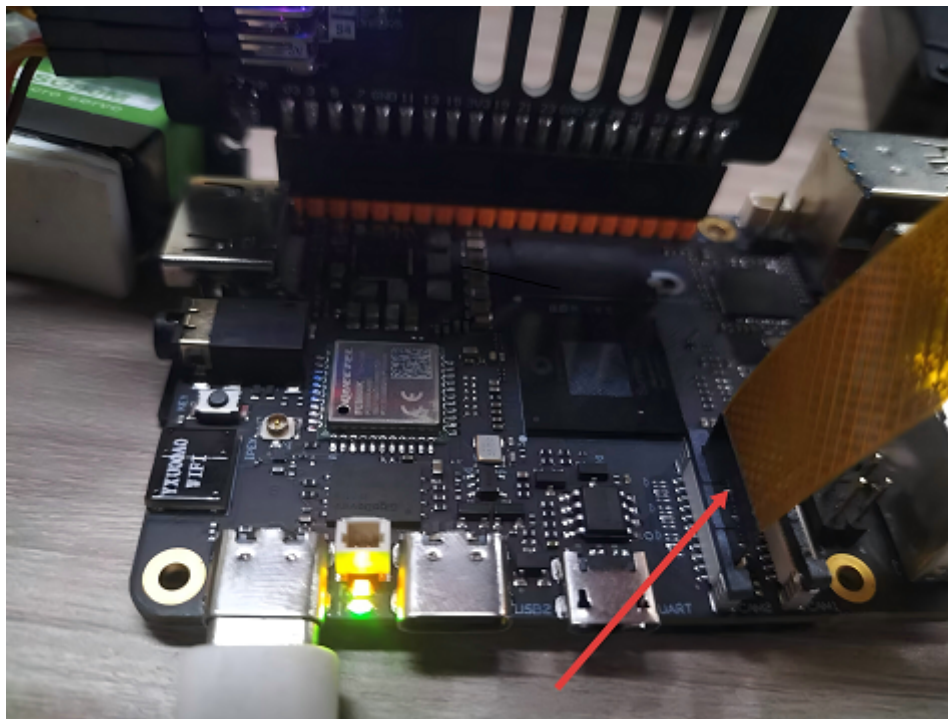
1. Tutorial Objectives

This tutorial demonstrates OpenClaw's natural language control capabilities through CSI camera peripherals. After completing this tutorial, readers will be able to:

- Understand the basic idea of using a camera for photography and visual analysis in OpenClaw
- Interact with OpenClaw via three methods: WhatsApp, TUI, and voice code
- Complete four camera cases: photo archiving, scene description, OCR recognition, target localization and face detection
- Understand the complete pipeline from "user proposes visual task" to "image capture" and "result return"

2. Hardware Preparation

- OpenClaw main board
- CSI camera module
- Camera ribbon cable
- [Optional] Microphone module
- Wiring diagram



3. Software Preparation

For international version:

If you are using WHATSAPP and want to upload images directly to WhatsApp, you need to fill in the conversation ID of your newly created WhatsApp conversation in the "allowFrom" path below.

- Open your newly created WhatsApp conversation to see the conversation ID.



- Fill this into "allowFrom":

```
{} openclaw.json X
.openclaw > {} openclaw.json > {} channels > {} feishu > abc chat_id
144   "commands": {
145     "native": "auto",
146     "nativeSkills": "auto",
147     "restart": true,
148     "ownerDisplay": "raw"
149   },
150   "session": {
151     "dmScope": "per-channel-peer"
152   },
153   "channels": {
154     "whatsapp": {
155       "enabled": true,
156       "dmPolicy": "allowlist",
157       "selfChatMode": true,
158       "allowFrom": [
159         "+8615338857526"
160       ],
161       "groupPolicy": "allowlist",
162       "debounceMs": 0,
163       "mediaMaxMb": 50
164     },
```

- Enter the following command in the terminal to take effect:

```
openclaw gateway restart
```

4. Testing Camera Application

Before starting this tutorial, please complete a basic `openclaw tui` conversation self-check first; if you haven't done so, see [05-Three Conversation Scheme Configuration and Testing]. Also, please confirm that the visual model has been configured, refer to [02-OpenClaw API-KEY Configuration] and [03-OpenClaw Model Switching]. After passing, proceed to the camera-specific test.

Enter the following command in the terminal to enter TUI:

openclaw tui

```
OpenClaw 2026.3.12 (6472949)
Your terminal just grew claws-type something and let the bot pinch the busywork.

openclaw tui - ws://127.0.0.1:18789 - agent main - session main
connecting | idle
agent main | session main | unknown | tokens ?

18:28:21 [plugins] feishu_chat: Registered feishu_chat, feishu_chat_members
18:28:21 [plugins] feishu_im: Registered feishu_im_user_message, feishu_im_user_fetch_resource, feishu_im_u
ser_get_messages, feishu_im_user_get_thread_messages, feishu_im_user_search_messages
18:28:21 [plugins] feishu_search: Registered feishu_search_doc_wiki
18:28:21 [plugins] feishu_drive: Registered feishu_drive_file, feishu_doc_comments, feishu_doc_media
18:28:21 [plugins] feishu_wiki: Registered feishu_wiki_space, feishu_wiki_space_node
18:28:21 [plugins] feishu_sheets: Registered feishu_sheet
18:28:21 [plugins] feishu_im: Registered feishu_im_bot_image
18:28:21 [plugins] Registered all OAPI tools (calendar, task, bitable, search, drive, wiki, sheets, im)
18:28:21 [plugins] feishu_doc: Registered feishu_fetch_doc, feishu_create_doc, feishu_update_doc
18:28:21 [plugins] feishu_oauth: Registered feishu_oauth tool

session agent:main:main
gateway connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens 0/1.0m (0%)
```

You can try entering the following content in the terminal:

- Help me take a photo
- Recognize the text in this image

The screenshot displays the OpenClaw TUI interface. On the left is a file explorer showing the directory structure, with the path `.openclaw > media > csi-camera > capture-20260427-190956.jpg` highlighted. The main area shows a video feed of a hand making a V-sign (peace sign) in an office setting. At the bottom, there is a chat window with the following text:

问题 输出 调试控制台 终端 窗口 1

需要我帮你开启高温提醒吗? (温度超过 30°C 时 RGB 红灯闪烁)

帮我拍一张照片

✅ 照片已拍摄

📷 摄像头成功捕捉到画面!

从照片中可以看到:

- 🖐️ 一个“V”字手势 (剪刀手/和平手势)
- 🏢 背景是办公室环境
- 💡 顶部有照明灯
- 📄 左侧似乎有文档或资料

照片已发送到当前会话。需要我帮你分析照片中的其他内容, 或者保存到 特定位置吗?

connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens ?/1.0m

- Find the cup in the frame (since the model will contact context, remind it to retake the photo, otherwise it might use the first photo)



- Retake a photo and organize the text inside into a table



5. Three Schemes for Conversing with OpenClaw

Scheme	Suitable Scenarios	Advantages	Suggested Positioning
WhatsApp	Remote control, classroom demonstration	Easy access, suitable for multi-user experience	Demonstrate "remote natural language control"
TUI	Local debugging, command verification	Most direct, easiest to troubleshoot	Get the visual pipeline running first
Voice Code	Voice interaction, complete AI experience	Closest to real conversation experience	As the final demonstration scheme

If you have selected the AI voice module + speaker, Feishu dialogue, tui, and WhatsApp dialogue need to broadcast the content of openclaw's reply. You can run the following program on the terminal. The AI voice module dialogue does not need to be run repeatedly

- Modify whether to enable broadcast parameter

```
/home/sunrise/voice_to_openclaw/.env
```

```
ENABLE_TTS=true # This parameter controls whether to broadcast. After modifying and saving, run the following program in the terminal.
```

- Enter in terminal:

```
cd ~/voice_to_openclaw
source .env/bin/activate
python -m src.openclaw_session_tts
```

```
(.venv) sunrise@ubuntu:~/voice_to_openclaw$ python -m src.openclaw_session_tts
[系统] OpenClaw 会话播报器已启动
[系统] 监听目录=/home/sunrise/.openclaw/agents/main/sessions
[系统] 监听渠道=feishu,tui,whatsapp
```

5.1 WhatsApp Scheme

For WhatsApp access steps, please refer directly to [06-Three Conversation Scheme Configuration and Testing]. This section only retains conversation examples suitable for the camera tutorial.

You can restart direct conversation, for example:

- See what is in front
- Recognize the text in the image

19:55



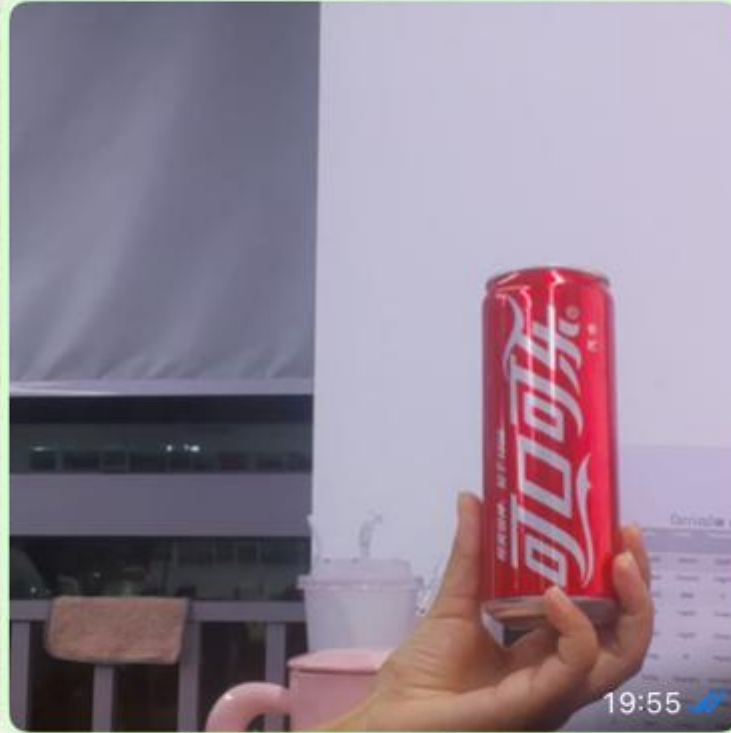
< 2



+86 153 3885 7526 (你)

给自己发消息

Look at what's in front of 19:55 ✓✓



I can see the scene in front of the camera:

- A hand is holding a **red Coca-Cola can** in the center
- To the right: a small **green desk fan** and a **water bottle**
- In the background: a **curriculum schedule** paper posted on the wall
- On the left: what looks like a **purple phone case** and a **window with blinds**
- Various items on the desk including a **pink mug** and some containers

The scene appears to be a desk or



5.2 TUI Scheme

If you have completed the TUI self-check in Part 6, you can continue the conversation by entering the following command in the terminal:

```
openclaw tui
```

Then you can directly input camera-related instructions, for example:

- Take a photo of the current scene
- Describe the scene in the lens
- Detect if there are any faces

You can add TUI conversation screenshots here.

5.3 Voice Code Scheme

Requires selection of AI voice interaction module + speaker

For voice module installation, environment configuration, and basic wake-up process, please refer directly to [04-AI Voice Interaction Configuration]. After wake-up, you can directly speak control content, for example:

- Help me take a photo
- See what is in front
- Help me read the text in the frame

```
[系统] 串口唤醒已启用: /dev/ttyUSB0@115200
[提示] 说“清空上下文”可以重置当前会话
[唤醒] 检测到串口信号, 开始新一轮对话
[听取] 请开始说话
[听取] 正在收音
[识别] 已捕获 6480 ms 语音, 正在转写

[你] Look at what's in front.
[发送] 正在交给 OpenClaw
[完成] OpenClaw 返回 200

[OpenClaw] I captured a photo! Here's what I see in front:
📷 **Scene
    Description:**

- A **hand making a peace sign (V sign)** in the
  center foreground
- A **water bottle** (怡寶 brand, 1.55L) on the
  right side with a green label
- A **window** with a gray
  curtain/blind partially drawn
- **Buildings visible outside** -
  looks like an office or residential complex
- Some items on a
  desk/shelf in the background

The view appears to be from an indoor
workspace looking out toward other buildings. Is there anything
specific you'd like me to analyze about this scene? 📷

[信息] 回复已裁剪为播报文本: 556 -> 121 字
/ [播报] 正在播放回复
```

You can add voice wake-up and voice control screenshots here.

6. Comprehensive Cases

Note: The following are conversation demonstrations conducted via `openClaw tui`. You can also choose WhatsApp or voice methods to complete them.

6.1 Case 1: Photo Archiving

Experiment Objective

Let OpenClaw complete the most basic photography function first and save the image.

Effect Description

After execution, a photo will be generated locally, usually keeping both the current photo file and the `latest.jpg` alias for the most recent photo. This case is suitable for verifying the image capture pipeline.

Example Instructions

- Help me take a photo
- Capture the current scene
- Take a photo now for record

6.2 Case 2: Scene Description

Experiment Objective

Let OpenClaw provide a brief description of the current scene after taking a photo.

Effect Description

OpenClaw can first complete the photography, then call the visual analysis script to make a one-sentence description of the image content, such as identifying desks, people, food, or common objects.

Example Instructions

- See what is in front
- Help me describe the current scene
- Take a photo and tell me what is mainly in the lens

6.3 Case 3: OCR Table Recognition

Experiment Objective

Let OpenClaw recognize text in the image and organize it into a table.

Effect Description

This case is suitable for photographing labels, cards, packaging, or simple documents, and then extracting the text. It can output plain text or continue to organize it into a table.

Example Instructions

- Recognize the text in this image
- Take a photo and organize the text into a table

6.4 Case 4: Target Localization and Face Detection

Experiment Objective

Let OpenClaw locate specific targets in the image, or detect if there are any faces in the frame. If a face is detected, the selected photo will default to:

```
/home/sunrise/.openc1aw/media/csi-camera/annotated/latest-face.png
```

Effect Description

This type of task is not just about "seeing," but further finding the position of target objects in the image, or recognizing faces and outputting annotation results. It is suitable for interactive displays and simple visual tasks.

Example Instructions

- Find the cup in the frame
- See if there are any faces
- Help me locate the mobile phone on the right side of the desk